



# Модуль 4. FastAPI: архитектура, жизненный цикл и обработка запросов

## Введение в модуль

После трёх модулей фундамента — Python, `asyncio`, сетевые протоколы — мы наконец подходим к фреймворку, ради которого создавался весь курс. Но FastAPI не будет казаться магией: мы знаем, что под ним лежит Starlette (ASGI), что ASGI стоит на event loop, а event loop — это кооперативная многозадачность над неблокирующими сокетами.

В этом модуле мы разберём:

- почему FastAPI построен именно так, и как его философия отличается от Django;
- что такое ASGI на уровне вызовов функций;
- как Pydantic V2 превращает аннотации типов в машину валидации со скоростью, близкой к компилируемому языку;
- как обрабатывать ошибки централизованно, не разбрасывая `try/except` по всем endpoint'ам;
- почему `BaseHTTPMiddleware` может убить производительность асинхронного сервера, и как писать «чистые» ASGI-мидлвари;
- как управлять жизненным циклом приложения, чтобы ML-модель загружалась один раз при старте и корректно освобождала память при остановке.

## 4.1. Философия фреймворка

### 4.1.1. Starlette + Pydantic = FastAPI

FastAPI — это не монолит. Это **композиция** двух специализированных инструментов:

| Компонент        | Отвечает за                                                                              | Аналогия          |
|------------------|------------------------------------------------------------------------------------------|-------------------|
| <b>Starlette</b> | ASGI-инфраструктуру: роутинг, middleware, lifespan, Request/Response объекты, WebSocket, | Двигатель и шасси |

| Компонент       | Отвечает за                              | Аналогия                            |
|-----------------|------------------------------------------|-------------------------------------|
|                 | фоновые задачи.                          | автомобиля.                         |
| <b>Pydantic</b> | Валидацию, сериализацию, генерацию схем. | Система безопасности и диагностики. |

FastAPI добавляет к Starlette:

- Автоматическую валидацию параметров через Pydantic.
- Автоматическую генерацию OpenAPI-схемы и документации Swagger UI / ReDoc.
- Dependency Injection систему ( Depends ).
- Нативную интеграцию с `async / await` без «костылей».

### 4.1.2. Type-driven development

В традиционном Python типы — это подсказки. В FastAPI типы — это **исполняемые спецификации**.

```
In [3]: from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    id: int
    name: str
    is_active: bool = True

@app.post("/users/")
async def create_user(user: User):
    return user
```

Что происходит, когда клиент отправляет `POST /users/` с JSON `{"id": "42", "name": "Alice"}`?

1. FastAPI видит аннотацию `user: User`.
2. Передаёт тело запроса в Pydantic.
3. Pydantic V2 (на Rust) парсит JSON, валидирует типы, приводит `"42"` к `int`, проверяет, что `name` не пустой.
4. Если валидация прошла — в функцию передаётся экземпляр `User`.
5. Если нет — автоматически возвращается `422 Unprocessable Entity` с детализацией ошибок.
6. OpenAPI-схема генерируется из полей `User` автоматически.

**Парадигма:** сначала типы, потом код. Функция не начинает выполняться, пока контракт не подтверждён.

### 4.1.3. Сравнение с Django REST Framework и Flask

| Аспект             | Django REST Framework                             | Flask + Flask-RESTX                                | FastAPI                                 |
|--------------------|---------------------------------------------------|----------------------------------------------------|-----------------------------------------|
| Асинхронность      | Частичная (с Django 4+), ORM не полностью async   | Нет (Werkzeug синхронен)                           | Нативная, на ASGI                       |
| Валидация          | Сериализаторы (классы с <code>validate_*</code> ) | <code>reqparse</code> или <code>marshmallow</code> | Pydantic, встроено                      |
| Документация       | Автогенерация через <code>get_schema_view</code>  | Swagger через аннотации                            | OpenAPI из коробки, без конфигурации    |
| Производительность | Средняя                                           | Низкая (синхронный WSGI)                           | Высокая (асинхронный, Pydantic на Rust) |
| Философия          | «Батарейки в комплекте» (monolith)                | Минимализм                                         | Type-driven, декларативность            |

Django — фреймворк «всё включено»: ORM, админка, аутентификация, формы. Это мощно для классических веб-приложений, но избыточно для ML-микросервисов, где нужен только JSON API.

Flask — минималистичный, но синхронный. Асинхронные расширения ( `Quart` ) — вторичны и фрагментированы.

FastAPI занимает нишу: **максимум производительности при минимуме кода** для API-сервисов.

## 4.2. ASGI: от WSGI к асинхронности

### 4.2.1. Почему WSGI мёртв для async

WSGI (Web Server Gateway Interface) — стандарт Python для веб-приложений с 2003 года. Его интерфейс:

```
In [ ]: def application(environ, start_response):  
        # environ: словарь с заголовками, методом, путём  
        # start_response: callable для установки статуса и заголовков  
        status = '200 OK'
```

```
headers = [('Content-Type', 'text/plain')]
start_response(status, headers)
return [b'Hello World']
```

Ключевая проблема: `application` — **синхронная функция**. Она должна вернуть итератор байтов немедленно. Она не может `await` чтения тела запроса или записи ответа.

Если внутри WSGI-приложения нужно сделать запрос к базе данных — весь `worker`-процесс блокируется. Gunicorn с синхронными `worker`'ами создаёт процесс на каждый запрос (или держит пул процессов). Это работает, но не масштабируется на десятки тысяч соединений.

## 4.2.2. ASGI-интерфейс: `scope`, `receive`, `send`

ASGI (Asynchronous Server Gateway Interface) — стандарт, принятый в 2016 году. Он описывает приложение как **асинхронный callable**:

```
In [ ]: async def application(scope, receive, send):
        ...
```

| Параметр             | Тип                                            | Назначение                                                                                                                                                                               |
|----------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>scope</code>   | <code>dict</code>                              | Контекст соединения: тип ( <code>http</code> / <code>websocket</code> ), метод, путь, заголовки, <code>query-string</code> , информация о сервере и клиенте. Живёт всё время соединения. |
| <code>receive</code> | <code>Callable[[], Awaitable[dict]]</code>     | Асинхронная функция для получения событий от клиента (тело запроса, сообщения <code>WebSocket</code> ).                                                                                  |
| <code>send</code>    | <code>Callable[[dict], Awaitable[None]]</code> | Асинхронная функция для отправки событий клиенту (старт ответа, тело, закрытие).                                                                                                         |

### Пример «голого» ASGI-приложения

```
In [ ]: async def app(scope, receive, send):
        # scope содержит метаданные о запросе
        assert scope['type'] == 'http'

        # Получаем тело запроса (если есть)
        body = b''
        while True:
            message = await receive() # Ждёт очередное сообщение от сервера
            if message['type'] == 'http.request': # Если это сообщение типа '
                body += message.get('body', b'')
                if not message.get('more_body', False): # Проверяет флаг more
                    # Если False — значит, это последний кусок, выходим из цикла
                    break
```

```

# В результате в переменной body будет полное тело запроса (байтовая)

# Формируем ответ
await send({
    'type': 'http.response.start', # тип сообщения: начало HTTP-ответ
    'status': 200,
    'headers': [(b'content-type', b'text/plain')], # список кортежей
})

await send({
    'type': 'http.response.body', # тип сообщения: само содержимое от
    'body': b'Hello, ASGI!',
})

```

Это и есть минимальное ASGI-приложение. Оно не использует FastAPI, Starlette или любой другой фреймворк — только протокол ASGI.

## Структура `scope` для HTTP

```

In [ ]: {
    'type': 'http',
    'asgi': {'version': '3.0', 'spec_version': '2.3'},
    'http_version': '1.1',
    'method': 'POST',
    'scheme': 'https',
    'path': '/users/',
    'query_string': b'page=1&limit=10',
    'root_path': '',
    'headers': [
        (b'host', b'api.example.com'),
        (b'content-type', b'application/json'),
        (b'content-length', b'47'),
    ],
    'client': ('192.168.1.1', 54321),
    'server': ('127.0.0.1', 8000),
}

```

```

In [ ]: {
    # === БАЗОВАЯ ИНФОРМАЦИЯ О ПРОТОКОЛЕ ===

    # Тип протокола. Может быть 'http', 'websocket', 'lifespan' и др.
    # Определяет, как обрабатывать запрос.
    'type': 'http',

    # Информация о версии ASGI, которую использует сервер.
    # 'version' - версия протокола ASGI
    # 'spec_version' - версия спецификации (для совместимости)
    'asgi': {'version': '3.0', 'spec_version': '2.3'},

    # Версия HTTP-протокола, которую использует клиент
    # Может быть '1.0', '1.1' или '2.0'
    'http_version': '1.1',
}

```

```

# === ИНФОРМАЦИЯ О ЗАПРОСЕ ===

# HTTP-метод (глагол) запроса
# Обычно: GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD
'method': 'POST',

# Схема (протокол) соединения
# 'http' - обычное соединение
# 'https' - зашифрованное (TLS/SSL)
'scheme': 'https',

# Путь запроса (без домена и параметров)
# Начинается с '/' и содержит маршрут до ресурса
'path': '/users/',

# Строка запроса (всё, что после '?' в URL)
# Хранится в виде байтов, т.к. может содержать любые символы
'query_string': b'page=1&limit=10',

# Корневой путь приложения (если приложение смонтировано не в корень)
# В большинстве случаев пустая строка
'root_path': '',

# === ЗАГОЛОВКИ ===

# Все HTTP-заголовки, переданные клиентом
# Каждый заголовок - кортеж (имя_в_байтах, значение_в_байтах)
# Все имена приведены к нижнему регистру
'headers': [
    (b'host', b'api.example.com'),      # Домен, к которому об
    (b'content-type', b'application/json'), # Тип содержимого (MIM
    (b'content-length', b'47'),          # Длина тела запроса в
],

# === ИНФОРМАЦИЯ О КЛИЕНТЕ ===

# IP-адрес и порт клиента (кто отправляет запрос)
'client': ('192.168.1.1', 54321),

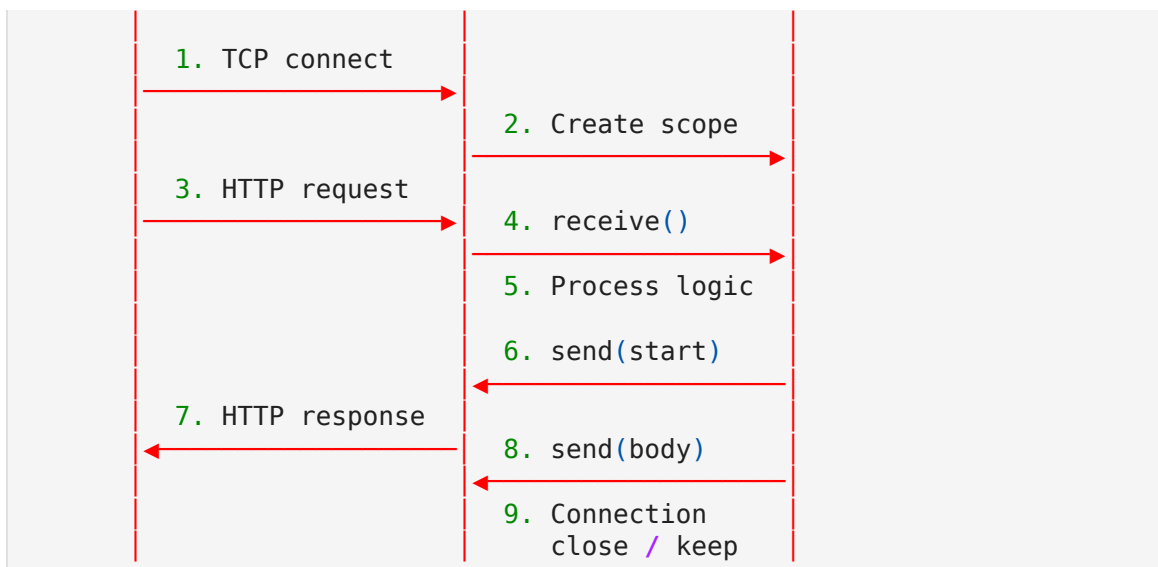
# IP-адрес и порт сервера (куда пришёл запрос)
'server': ('127.0.0.1', 8000),
}

```

### 4.2.3. Жизненный цикл запроса в ASGI

In [ ]:





Критически важно: `receive` и `send` — это **не просто функции**. Они — каналы связи между сервером (Uvicorn) и приложением (FastAPI). Uvicorn читает байты из сокета, парсит HTTP, упаковывает в события и передаёт через `receive`. FastAPI формирует ответ, передаёт через `send`, а Uvicorn сериализует в HTTP и пишет в сокет.

#### 4.2.4. ASGI-серверы: Uvicorn, Hypercorn, Daphne

| Сервер           | Особенности                                                                                                              | Когда использовать             |
|------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b>Uvicorn</b>   | Быстрый, на <code>uvloop</code> (Cython-реализация event loop), поддерживает HTTP/1.1 и WebSocket. Стандарт для FastAPI. | Почти всегда.                  |
| <b>Hypercorn</b> | Поддерживает HTTP/2, HTTP/3, WebSocket. Написан на чистом Python, медленнее Uvicorn.                                     | Когда нужен HTTP/2 или HTTP/3. |
| <b>Daphne</b>    | Первый ASGI-сервер, разработан для Django Channels.                                                                      | Legacy, Django Channels.       |

Uvicorn использует `uvloop` — реализацию event loop на Cython, которая в 2-4 раза быстрее стандартного `asyncio`. Он также использует `httptools` — парсер HTTP на C.

```

In [ ]: # Запуск
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4

# main:app — объект FastAPI в файле main.py
# --workers 4 — 4 процесса-воркера (multiprocessing)

```

## 4.2.5. Математическая подоплека: ASGI как система ввода-вывода с конечным автоматом

С точки зрения теории автоматов, ASGI-приложение для HTTP-запроса — это автомат с состояниями:

```
$$Q = \{ \text{INIT}, \text{RECEIVING}, \text{PROCESSING}, \text{SENDING}, \text{DONE} \}$$
```

Переходы:

- $\text{INIT} \rightarrow \text{RECEIVING}$  (создан)
- $\text{RECEIVING} \rightarrow \text{RECEIVING}$  (если `more_body=True`)
- $\text{RECEIVING} \rightarrow \text{PROCESSING}$  (если `more_body=False`)
- $\text{PROCESSING} \rightarrow \text{SENDING}$
- $\text{SENDING} \rightarrow \text{DONE}$  (если `more_body=False`)

Uvicorn управляет множеством таких автоматов конкурентно, используя event loop.

## 4.3. Маршрутизация и обработчики

### 4.3.1. APIRouter: модульность приложения

Монолитный `main.py` с сотнями endpoint'ов не масштабируется. FastAPI использует `APIRouter` для разбиения на модули:

```
In [ ]: # routers/users.py
from fastapi import APIRouter

router = APIRouter(
    prefix="/users",
    tags=["users"],
    responses={404: {"description": "Not found"}},
)

@router.get("/") # реальный путь будет /users/
async def list_users():
    return [{"id": 1, "name": "Alice"}]

@router.get("/{user_id}") # Реальный путь: /users/{user_id} --> запросы т
```



```

async def get_user(user_id: int):
    return {"id": user_id, "name": "Alice"}

```

```

In [ ]: # main.py
from fastapi import FastAPI
from routers.users import router as users_router

app = FastAPI()
app.include_router(users_router)

```

`prefix="/users"` означает, что все пути в роутере автоматически получают префикс. `tags=["users"]` группирует endpoint'ы в документации Swagger UI.

| Параметр                            | Назначение                                                     | Пример                                                                                            |
|-------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>prefix</code>                 | Добавляет префикс ко всем путям роутера                        | Если указать <code>/users</code> , то <code>@router.get("/")</code> --> путь <code>/users/</code> |
| <code>tags</code>                   | Группирует эндпоинты в Swagger UI для удобства навигации       | Отобразится отдельная секция "users"                                                              |
| <code>responses</code>              | Описывает общие возможные ответы для всех эндпоинтов в роутере | В документации добавится описание ошибки 404                                                      |
| <code>dependencies</code>           | Общие зависимости для всех эндпоинтов                          | Авторизация, логирование                                                                          |
| <code>default_response_class</code> | Класс ответа по умолчанию                                      | JSONResponse, HTMLResponse и др.                                                                  |

## 4.3.2. Path parameters: типизация и валидация

```

In [ ]: @app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}

```

FastAPI извлекает `item_id` из пути, **парсит** его как `int` и передаёт в функцию. Если передать `/items/abc` — автоматический ответ `422 Unprocessable Entity` с детализацией.

Можно использовать встроенные типы путей:

```

In [ ]: @app.get("/files/{file_path:path}")
async def read_file(file_path: str):
    return {"file_path": file_path}

```

`:path` означает, что сегмент может содержать `/`. Без этого `/files/a/`

b.txt не сматчится.

### 4.3.3. Query parameters: валидация через Query

```
In [5]: from fastapi import Query

@app.get("/items/")
async def read_items(
    q: str | None = Query(default=None, min_length=3, max_length=50),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=10, ge=1, le=100),
):
    return {"q": q, "skip": skip, "limit": limit}

await read_items()
```

```
Out[5]: {'q': Query(None), 'skip': Query(0), 'limit': Query(10)}
```

`Query(...)` — это не просто значение по умолчанию. Это **декларатор**, который сообщает FastAPI:

- Этот параметр приходит из query-string ( `?skip=0&limit=10` ).
- У него есть ограничения: `ge` (greater or equal), `le` (less or equal), `min_length`, `regex` и др.

Если клиент отправит `?limit=200`, FastAPI вернёт `422` с сообщением: «limit must be less than or equal to 100».

| Параметр<br>Query() | Назначение                                          | Пример                                    |
|---------------------|-----------------------------------------------------|-------------------------------------------|
| <b>default</b>      | Значение по умолчанию (если параметр не передан)    | <code>Query(1)</code> — по умолчанию 1    |
| <b>description</b>  | Описание в Swagger UI                               | <code>description="Номер страницы"</code> |
| <b>ge</b>           | Greater or Equal — минимальное значение (для чисел) | <code>ge=1</code> — значение $\geq 1$     |
| <b>le</b>           | Less or Equal — максимальное значение               | <code>le=100</code> — значение $\leq 100$ |
| <b>gt</b>           | Greater Than — строго больше                        | <code>gt=0</code> — значение $> 0$        |
| <b>lt</b>           | Less Than — строго меньше                           | <code>lt=100</code> — значение $< 100$    |
| <b>min_length</b>   | Минимальная длина строки                            | <code>min_length=3</code>                 |
| <b>max_length</b>   | Максимальная длина строки                           | <code>max_length=50</code>                |
| <b>regex</b>        | Регулярное выражение для                            | <code>regex=r"^[a-z]+\$"</code>           |

| Параметр<br>Quary() | Назначение                       | Пример                                                             |
|---------------------|----------------------------------|--------------------------------------------------------------------|
|                     | проверки                         |                                                                    |
| <b>example</b>      | Пример значения в документации   | example=1                                                          |
| <b>alias</b>        | Альтернативное имя параметра     | Если в URL <code>?user-id=5</code> , а в коде <code>user_id</code> |
| <b>deprecated</b>   | Пометить параметр как устаревший | deprecated=True                                                    |

#### 4.3.4. Request Body: Pydantic модели

```
In [ ]: from pydantic import BaseModel, Field

class Item(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    description: str | None = None
    price: float = Field(gt=0, description="Price must be greater than zero")
    tax: float | None = None

@app.post("/items/")
async def create_item(item: Item):
    return item
```

| Параметр Field()               | Назначение                                | Пример                         |
|--------------------------------|-------------------------------------------|--------------------------------|
| <b>default</b>                 | Значение по умолчанию                     | Field("guest")                 |
| <b>...</b> (три точки)         | Обязательное поле (не может быть None)    | Field(...)                     |
| <b>description</b>             | Описание в Swagger UI                     | description="Имя пользователя" |
| <b>ge / le</b>                 | Минимальное/максимальное значение (числа) | ge=0, le=120                   |
| <b>gt / lt</b>                 | Строго больше/меньше                      | gt=0                           |
| <b>min_length / max_length</b> | Длина строки                              | min_length=3, max_length=50    |
| <b>pattern</b>                 | Регулярное выражение (Python 3.10+)       | pattern=r"^[a-z]+\$"           |
| <b>example</b>                 | Пример в документации                     | example="alex123"              |
| <b>alias</b>                   | Альтернативное имя поля                   | Если JSON с ключом "user-name" |
| <b>title</b>                   | Заголовок поля в документации             | title="Имя пользователя"       |
| <b>deprecated</b>              | Пометить поле как устаревшее              | deprecated=True                |
| <b>exclude</b>                 | Исключить поле при                        | exclude=True                   |

| Параметр Field()            | Назначение                                   | Пример                                                              |
|-----------------------------|----------------------------------------------|---------------------------------------------------------------------|
| сериализации                |                                              |                                                                     |
| Инструмент                  | Где используется                             | Для чего                                                            |
| <code>fastapi.Query</code>  | В параметрах <b>функции</b> <b>эндпоинта</b> | Валидация <b>query-параметров</b> (часть URL после <code>?</code> ) |
| <code>pydantic.Field</code> | Внутри <b>Pydantic-схем</b> (моделей)        | Валидация <b>полей модели</b> (тело запроса, ответы)                |

### Вложенные модели:

```
In [ ]: class Image(BaseModel):
        url: str
        name: str

        class Item(BaseModel):
            name: str
            images: list[Image] = []
```

FastAPI автоматически ожидает JSON:

```
In [ ]: {
        "name": "Foo",
        "images": [
            {"url": "http://example.com/1.jpg", "name": "img1"}
        ]
    }
```

## 4.3.5. Заголовки, куки, файлы, формы

### Заголовки:

```
In [ ]: from fastapi import Header

@app.get("/items/")
async def read_items(user_agent: str | None = Header(default=None)):
    return {"User-Agent": user_agent}
```

### Куки:

```
In [ ]: from fastapi import Cookie

@app.get("/items/")
async def read_items(session_id: str | None = Cookie(default=None)):
    return {"session_id": session_id}
```

### Файлы:

```
In [ ]: from fastapi import File, UploadFile
```

```
@app.post("/upload/")
async def upload_file(file: UploadFile = File(...)):
    content = await file.read()
    return {"filename": file.filename, "size": len(content)}
```

`UploadFile` — это обёртка над `SpooledTemporaryFile`: маленькие файлы хранятся в памяти, большие — на диске. `await file.read()` — асинхронное чтение, не блокирующее event loop.

### Формы:

```
In [ ]: from fastapi import Form
```

```
@app.post("/login/")
async def login(username: str = Form(...), password: str = Form(...)):
    return {"username": username}
```

```
In [ ]: Отлично! Давайте разберём **все** основные инструменты FastAPI для получе
```

---

## 1. Общая схема: откуда берутся данные

...

HTTP-запрос

|                   |                          |
|-------------------|--------------------------|
| URL               |                          |
| Path (путь)       | → Path()                 |
| Query (параметры) | → Query()                |
| ... (фрагменты)   |                          |
| Заголовки         | → Header()               |
| Тело (Body)       |                          |
| JSON / Pydantic   | → Body() / автоматически |
| Form-данные       | → Form()                 |
| Файлы             | → File() / UploadFile    |
| Куки (Cookie)     | → Cookie()               |

...

---

## 2. `Path()` — параметры пути

Параметры, которые являются частью URL-пути (в `{}`).

### Синтаксис

```
```python
from fastapi import Path

@router.get("/users/{user_id}")
```

```

async def get_user(
    user_id: int = Path(..., description="ID пользователя", ge=1)
):
    return {"user_id": user_id}

# Запрос: GET /users/42
# user_id = 42
'''

### Основные параметры

| Параметр | Назначение |
|-----|-----|
| `ge` / `le` | Минимальное/максимальное значение |
| `gt` / `lt` | Строго больше/меньше |
| `description` | Описание в документации |
| `example` | Пример значения |

### Особенности

- **Всегда обязательный** (без значения путь не сопадёт)
- Можно использовать `Path(...)` или просто указать тип — FastAPI поймёт:

'''python
# Эти два варианта эквивалентны:
async def get_user(user_id: int): ... # FastAPI автоматически понимает,
async def get_user(user_id: int = Path(...)): ... # Явное указание
'''

---

## 3. `Query()` — параметры строки запроса

Параметры после `?` в URL. Мы уже разбирали подробно.

'''python
from fastapi import Query

@router.get("/users")
async def get_users(
    page: int = Query(1, ge=1, description="Номер страницы"),
    limit: int = Query(10, ge=1, le=100, description="Записей на странице"),
    search: str = Query(None, description="Поисковый запрос")
):
    return {"page": page, "limit": limit, "search": search}

# Запрос: GET /users?page=2&limit=20&search=alex
'''

---

## 4. `Header()` — заголовки запроса

Извлекает значения из HTTP-заголовков.

```

```

python
from fastapi import Header

@router.get("/protected")
async def protected_route(
    authorization: str = Header(..., description="Токен авторизации"),
    user_agent: str | None = Header(None, description="Браузер пользователя"),
    x_request_id: str = Header(None, alias="X-Request-ID") # Кастомный з
):
    return {
        "token": authorization,
        "user_agent": user_agent,
        "request_id": x_request_id
    }

# Запрос с заголовками:
# Authorization: Bearer token123
# User-Agent: Mozilla/5.0
# X-Request-ID: abc-123

```

### Ключевые особенности

| Особенность                       | Пояснение                                   |
|-----------------------------------|---------------------------------------------|
| **Автоматическое преобразование** | `user-agent` → `user_agent` (дефисы)        |
| **`alias`**                       | Для нестандартных заголовков с дефисами     |
| **Регистронезависимость**         | Заголовки `Authorization` и `authorization` |

---

## 5. `Cookie()` – куки

Извлекает значения из cookies.

```

python
from fastapi import Cookie

@router.get("/session")
async def get_session(
    session_id: str = Cookie(..., description="ID сессии"),
    user_preferences: str | None = Cookie(None, description="Настройки")
):
    return {
        "session_id": session_id,
        "preferences": user_preferences
    }

# Клиент отправляет Cookie: session_id=abc123; user_preferences=dark_mode

```

### Особенности

- Куки автоматически парсятся из заголовка `Cookie`
- Если куки нет — вернётся значение по умолчанию (или ошибка, если `...`)

---

## 6. `Body()` — тело запроса (JSON)

Используется для получения данных из тела запроса. FastAPI \*\*автоматически

```

python
from fastapi import Body
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    email: str

@router.post("/users")
async def create_user(
    user: UserCreate, # FastAPI автоматически читает из Body
):
    return user

# Явное использование Body (для дополнительных параметров)
@router.post("/items")
async def create_item(
    item: dict = Body(..., description="Данные товара"),
    priority: int = Body(1, description="Приоритет", ge=0)
):
    return {"item": item, "priority": priority}

```

### Параметры `Body()`

| Параметр      | Назначение                                       |
|---------------|--------------------------------------------------|
| `description` | Описание в документации                          |
| `example`     | Пример значения                                  |
| `embed`       | Если нужно передать одно значение внутри объекта |

### `embed=True` — важный нюанс

```

python
@router.post("/items")
async def create_item(
    name: str = Body(..., embed=True) # Ожидает {"name": "value"}
):
    return {"name": name}

# Без embed: передаётся просто "value"
# С embed: передаётся {"name": "value"}

```

In [ ]: # `Form()` — данные из HTML-форм



```

# Используется для получения данных из `application/x-www-form-urlencoded`

from fastapi import Form

@router.post("/login")
async def login(
    username: str = Form(..., description="Имя пользователя"),
    password: str = Form(..., description="Пароль"),
    remember_me: bool = Form(False, description="Запомнить меня")
):
    return {
        "username": username,
        "remember_me": remember_me
    }

# HTML-форма:
# <form method="post" action="/login">
#   <input name="username">
#   <input name="password" type="password">
#   <input name="remember_me" type="checkbox">
# </form>

### Ключевые моменты

#- Используется **только** с POST-запросами
#- Автоматически парсит `application/x-www-form-urlencoded`
#- Не работает с JSON

```

In [ ]: #`File()` – просто байты

```

from fastapi import File

@router.post("/upload")
async def upload_file(
    file: bytes = File(..., description="Файл для загрузки")
):
    return {"size": len(file)}

```

In [ ]: #`UploadFile` – продвинутая работа с файлами

```

from fastapi import UploadFile

@router.post("/upload")
async def upload_file(
    file: UploadFile = File(..., description="Файл для загрузки")
):
    # Метаданные файла
    print(file.filename)      # Имя файла
    print(file.content_type)  # MIME-тип (image/png, application/pdf)
    print(file.size)          # Размер в байтах

    # Чтение содержимого (асинхронно)

```

```

contents = await file.read()

# Сохранение на диск
with open(f"uploads/{file.filename}", "wb") as f:
    f.write(contents)

# Закрывать файл
await file.close()

return {"filename": file.filename}

```

```

In [ ]: #Загрузка нескольких файлов

from typing import List

@router.post("/upload-multiple")
async def upload_multiple(
    files: List[UploadFile] = File(..., description="Список файлов")
):
    return {
        "filenames": [file.filename for file in files],
        "count": len(files)
    }

```

## Сравнение bytes vs UploadFile

| Критерий           | bytes = File()               | UploadFile = File()                                |
|--------------------|------------------------------|----------------------------------------------------|
| Размер             | Загружается целиком в память | Читается по частям (эффективно для больших файлов) |
| Метаданные         | Нет (только содержимое)      | Есть (filename, content_type, size)                |
| Асинхронность      | Нет                          | Да (async)                                         |
| Когда использовать | Маленькие файлы (< 1 МБ)     | Большие файлы, потоковая обработка                 |

## Сводная таблица всех инструментов

| Инструмент | Часть запроса            | Тип контента     | Обязательность                  |
|------------|--------------------------|------------------|---------------------------------|
| Path()     | URL<br>( /users/{id} ) - |                  | Всегда обязателен               |
| Query()    | URL<br>( ?key=value ) -  |                  | Опционально (если есть default) |
| Header()   | Заголовки -              |                  | Опционально                     |
| Cookie()   | Cookies -                |                  | Опционально                     |
| Body()     | Тело запроса             | application/json | Опционально                     |

| Инструмент              | Часть запроса | Тип контента                                   | Обязательность |
|-------------------------|---------------|------------------------------------------------|----------------|
| <code>Form()</code>     | Тело запроса  | <code>application/x-www-form-urlencoded</code> | Опционально    |
| <code>File()</code>     | Тело запроса  | <code>multipart/form-data</code>               | Опционально    |
| <code>UploadFile</code> | Тело запроса  | <code>multipart/form-data</code>               | Опционально    |

## 4.4. Pydantic V2: модели данных и внутреннее устройство

### 4.4.1. От V1 к V2: почему это революция

Pydantic V1 (2019–2023) был написан на чистом Python. Валидация больших моделей была узким местом: десериализация JSON в Python-объекты с проверкой типов могла занимать миллисекунды на запрос. При тысячах RPS это становится критичным.

Pydantic V2 (2023+) переписал ядро на **Rust** ( `pydantic-core` ). Rust даёт:

- Отсутствие GIL (Global Interpreter Lock): валидация выполняется вне Python-интерпретатора.
- Zero-сору парсинг: где возможно, данные не копируются, а ссылаются на исходный буфер.
- Компиляцию в машинный код: нет накладных расходов интерпретации.

**Сравнение производительности** (примерные цифры, зависят от железа):

| Операция                 | Pydantic V1 | Pydantic V2    | Стандартный <code>json.loads</code> |
|--------------------------|-------------|----------------|-------------------------------------|
| Валидация сложной модели | 1.0× (база) | 5–20× быстрее  | Н/Д (нет валидации)                 |
| Сериализация в JSON      | 1.0×        | 10–50× быстрее | 0.5× V2 (нет валидации)             |

### 4.4.2. Базовая модель

```
In [7]: from pydantic import BaseModel, Field, ConfigDict

class User(BaseModel):
```

```

model_config = ConfigDict(strict=False) # аналог class Config в V1

id: int
name: str = Field(min_length=1)
email: str
age: int = Field(ge=0, le=150)

```

### Ключевые методы:

```

In [11... user = User(id="42", name="Alice", email="a@b.com", age=30)

# Сериализация
user.model_dump()           # dict
user.model_dump_json()      # str (JSON)
user.model_dump(mode="json") # dict с JSON-совместимыми типами (datetime

```

```

Out[11]: {'id': 42, 'name': 'Alice', 'email': 'a@b.com', 'age': 30}

```

```

In [12... # Валидация из JSON
json_str = '{"id": 42, "name": "Alice", "email": "a@b.com", "age": 30}'
user = User.model_validate_json(json_str)
user

```

```

Out[12]: User(id=42, name='Alice', email='a@b.com', age=30)

```

```

In [14... # Валидация из dict
data = {"id": 42, "name": "Alice", "email": "a@b.com", "age": 30}
user = User.model_validate(data)
user

```

```

Out[14]: User(id=42, name='Alice', email='a@b.com', age=30)

```

## 4.4.3. Валидаторы: field\_validator и model\_validator

**Field validator** — проверка отдельного поля:

```

In [ ]: from pydantic import BaseModel, field_validator

class User(BaseModel):
    email: str

    @field_validator('email')
    @classmethod
    def validate_email(cls, v: str) -> str:
        if '@' not in v:
            raise ValueError('Invalid email')
        return v.lower()

```

**Model validator** — проверка всей модели:

```
In [ ]: from pydantic import BaseModel, model_validator

class Rectangle(BaseModel):
    width: float
    height: float

    @model_validator(mode='after')
    def check_dimensions(self):
        if self.width <= 0 or self.height <= 0:
            raise ValueError('Dimensions must be positive')
        return self
```

Режимы:

- `mode='after'` — валидатор вызывается после базовой валидации полей (поля уже приведены к типам).
- `mode='before'` — вызывается до валидации, получает сырые данные (dict).

#### 4.4.4. Внутреннее устройство `pydantic-core`

`pydantic-core` — это Rust-библиотека, к которой Pydantic V2 обращается через Python C-API / PyO3.

Архитектура:

```
In [ ]: 
```

|                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------|
| Python Layer<br>BaseModel, Field, ConfigDict,<br>field_validator, model_validator                                                      |
| Rust Core (pydantic-core)<br>- JSON Parser (serde_json)<br>- Schema Builder<br>- Validator Engine<br>- Type Coercion (int<->str, etc.) |
| Python C-API / PyO3<br>Передача данных между Python и Rust                                                                             |

**Zero-copy парсинг:**

Когда `model_validate_json()` получает строку JSON, Rust-парсер `serde_json` десериализует её в Rust-структуры. Затем, если типы

совпадают (например, JSON-строка -> Python `str`, JSON-число -> Python `int`), данные **переносятся** в Python-объекты без промежуточного копирования. Это возможно благодаря тому, что Rust и Python C-API работают с непрерывными блоками памяти.

### Strict mode vs Lax mode:

```
In [ ]: class User(BaseModel):
        model_config = ConfigDict(strict=True)
        id: int

# strict=True: "42" вызовет ошибку (str не int)
# strict=False (по умолчанию): "42" автоматически преобразуется в 42
```

## 4.4.5. Математическая подоплека: JSON Schema как формальная грамматика

JSON Schema — это язык описания структуры JSON-документов. Pydantic автоматически генерирует JSON Schema для каждой модели:

```
In [ ]: print(User.model_json_schema())
```

Результат:

```
In [ ]: {
    "title": "User",
    "type": "object",
    "properties": {
        "id": {"type": "integer"},
        "name": {"type": "string", "minLength": 1},
        "email": {"type": "string", "format": "email"},
        "age": {"type": "integer", "minimum": 0, "maximum": 150}
    },
    "required": ["id", "name", "email", "age"]
}
```

Формально, JSON Schema определяет **язык**  $L$  — множество всех JSON-документов, удовлетворяющих схеме. Валидация Pydantic — это **распознавание принадлежности** входного документа  $d$  языку  $L$ :

$$\text{validate}(d) = \begin{cases} \text{OK}, & d \in L \\ \text{Error}, & d \notin L \end{cases}$$

JSON Schema не является полным по Тьюрингу (это хорошо): проверка принадлежности всегда завершается за конечное время. Сложность валидации линейна относительно размера входа  $O(n)$ , что достигается за счёт однопроходного парсинга в Rust.

## 4.5. Ответы клиенту

### 4.5.1. Типы Response

FastAPI (через Starlette) предоставляет несколько классов ответов:

```
In [ ]: from fastapi import Response, JSONResponse, HTMLResponse, StreamingResponse
```

| Класс             | Назначение                     | Content-Type               |
|-------------------|--------------------------------|----------------------------|
| Response          | Базовый, сырые байты           | text/plain                 |
| JSONResponse      | Сериализация dict/list в JSON  | application/json           |
| HTMLResponse      | HTML-строка                    | text/html                  |
| PlainTextResponse | Текст                          | text/plain                 |
| StreamingResponse | Потоковая передача (генератор) | задаётся явно              |
| FileResponse      | Отправка файла с диска         | определяется по расширению |

### 4.5.2. Возврат моделей Pydantic

По умолчанию FastAPI сериализует возвращаемое значение в JSON:

```
In [ ]: @app.get("/user/")
async def get_user() -> User:
    return User(id=1, name="Alice", email="a@b.com", age=30)
```

FastAPI вызывает `jsonable_encoder()` — функцию, которая рекурсивно преобразует:

- Pydantic-модели -> dict
- `datetime` -> ISO-8601 строку
- `UUID` -> строку
- `set` -> list
- `Enum` -> значение

А затем `json.dumps()`.

### 4.5.3. StreamingResponse

Критически важен для ML-сервисов, генерирующих данные по частям

(токены текста, чанки аудио):

```
In [ ]: from fastapi import FastAPI
        from fastapi.responses import StreamingResponse

        app = FastAPI()

        async def token_generator():
            tokens = ["Hello", " world", " from", " ML", " model", "!"]
            for token in tokens:
                yield token
                await asyncio.sleep(0.1) # имитация генерации

        @app.get("/stream")
        async def stream_tokens():
            return StreamingResponse(token_generator(), media_type="text/plain")
```

Под капотом `StreamingResponse`:

1. Отправляет `http.response.start` со статусом 200.
2. В цикле `async for` читает чанки из генератора.
3. Отправляет `http.response.body` с `more_body=True` для каждого чанка.
4. Последний чанк отправляет с `more_body=False`.

#### 4.5.4. FileResponse

```
In [ ]: from fastapi.responses import FileResponse

        @app.get("/download/{filename}")
        async def download_file(filename: str):
            return FileResponse(
                path=f"/storage/{filename}",
                filename=filename,
                media_type="application/octet-stream"
            )
```

`FileResponse` использует `sendfile()` системный вызов (где доступен), который копирует данные из файла в сокет **в обход пользовательского пространства** (zero-copy). Это существенно снижает нагрузку на CPU при передаче больших файлов.

### 4.6. Обработка ошибок и исключений



## 4.6.1. HTTPException

FastAPI предоставляет стандартный механизм для HTTP-ошибок:

```
In [ ]: from fastapi import HTTPException

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id < 0:
        raise HTTPException(status_code=400, detail="Item ID must be posi
    return {"item_id": item_id}
```

`HTTPException` — это не обычное Python-исключение. Это **сигнал** для FastAPI: «сформируй HTTP-ответ с этим статусом и телом». Она не прерывает выполнение event loop, а лишь меняет путь обработки запроса.

Можно добавить заголовки:

```
In [ ]: raise HTTPException(
    status_code=401,
    detail="Unauthorized",
    headers={"WWW-Authenticate": "Bearer"}
)
```

## 4.6.2. Кастомные Exception Handlers

В большом приложении разбрасывать `HTTPException` по endpoint'ам — антипаттерн. Бизнес-логика должна выбрасывать **свои** исключения, а фреймворк — превращать их в HTTP-ответы.

### Шаг 1: определяем иерархию исключений

```
In [ ]: class AppException(Exception):
    """Базовое исключение приложения"""
    def __init__(self, message: str, status_code: int = 500):
        self.message = message
        self.status_code = status_code
        super().__init__(self.message)

    class NotFoundError(AppException):
        def __init__(self, resource: str, resource_id: str):
            super().__init__(
                message=f"{resource} with id={resource_id} not found",
                status_code=404
            )
```

```
class ValidationError(AppException):
    def __init__(self, field: str, reason: str):
        super().__init__(
            message=f"Validation failed for {field}: {reason}",
            status_code=422
        )
```

## Шаг 2: регистрируем обработчики

```
In [ ]: from fastapi import Request
        from fastapi.responses import JSONResponse

        @app.exception_handler(AppException)
        async def app_exception_handler(request: Request, exc: AppException):
            return JSONResponse(
                status_code=exc.status_code,
                content={"error": exc.message, "type": exc.__class__.__name__}
            )

        @app.exception_handler(NotFoundError)
        async def not_found_handler(request: Request, exc: NotFoundError):
            # Можно добавить логирование, метрики
            return JSONResponse(
                status_code=404,
                content={"error": exc.message, "resource": "unknown"}
            )
```

## Шаг 3: используем в бизнес-логике

```
In [ ]: @app.get("/users/{user_id}")
        async def get_user(user_id: str):
            user = await user_repository.find_by_id(user_id)
            if user is None:
                raise NotFoundError("User", user_id)
            return user
```

### Преимущества:

- Endpoint'ы не знают о HTTP. Они работают с бизнес-исключениями.
- Централизованная обработка: можно добавить логирование, трассировку, метрики в одном месте.
- Иерархия: `AppException` ловит все производные, если нет более специфичного handler'а.

## 4.6.3. Глобальная обработка и логирование

```
In [ ]: import logging
        import traceback
```

```

logger = logging.getLogger("app")

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    # Логируем непредвиденные ошибки
    logger.error(
        f"Unhandled exception: {exc}\n{traceback.format_exc()}",
        extra={"path": request.url.path, "method": request.method}
    )

    return JSONResponse(
        status_code=500,
        content={"error": "Internal server error", "request_id": correlat
    )

```

**Важно:** handler для `Exception` — ловушка последней инстанции. Она должна возвращать **generic** сообщение, чтобы не раскрывать внутренние детали (пути файлов, структуру БД) злоумышленнику.

## 4.7. Middleware: BaseHTTPMiddleware vs чистые ASGI-мидлвари

### 4.7.1. Зачем нужны middleware?

Middleware — это **конвейер** обработки запроса. Каждый middleware может:

- модифицировать входящий запрос (добавить заголовки, распарсить токен);
- выполнить код до и после endpoint'a (тайминг, логирование);
- прервать запрос (аутентификация, rate limiting);
- модифицировать ответ (добавить CORS-заголовки, сжатие).

### 4.7.2. BaseHTTPMiddleware: простота и скрытая опасность

```

In [ ]: from fastapi import Request
        from starlette.middleware.base import BaseHTTPMiddleware
        import time

        class TimingMiddleware(BaseHTTPMiddleware):
            async def dispatch(self, request: Request, call_next):
                start = time.perf_counter()
                response = await call_next(request) # <- вызов следующего middle
                elapsed = time.perf_counter() - start
                response.headers["X-Process-Time"] = str(elapsed)

```

```
        return response

app.add_middleware(TimingMiddleware)
```

`BaseHTTPMiddleware` удобен, потому что предоставляет готовые объекты `Request` и `Response`. Но у него есть **фатальный недостаток** для асинхронных приложений.

### Почему BaseHTTPMiddleware блокирует event loop?

`BaseHTTPMiddleware` оборачивает ASGI-приложение в **синхронный** вызов `call_next`. Под капотом он использует `asyncio.run_coroutine_threadsafe()` или похожий механизм, который создаёт **новую задачу** в event loop и ждёт её через `asyncio.Queue`.

Это означает:

- Весь response body собирается во временный буфер перед передачей клиенту.
- **StreamingResponse теряет смысл**: чанки накапливаются в памяти, а не отправляются по мере готовности.
- При больших ответах происходит **избыточное потребление памяти**.
- В некоторых сценариях возможен **deadlock**, если middleware вызывает `call_next`, а endpoint пытается отправить данные, но middleware ждёт завершения `call_next`.

**Вывод:** `BaseHTTPMiddleware` приемлем для простых middleware (логирование заголовков, добавление заголовков ответа), но **опасен** для streaming, больших файлов и высоконагруженных систем.

### 4.7.3. Чистые ASGI-мидлвари: полный контроль

Чистая ASGI-middleware — это функция (или класс), которая оборачивает другое ASGI-приложение, перехватывая `scope`, `receive`, `send`.

```
In [ ]: class ASGITimingMiddleware:
        def __init__(self, app):
            self.app = app # следующее приложение в цепочке

        async def __call__(self, scope, receive, send):
            if scope["type"] != "http":
                # WebSocket или lifespan пропускаем без изменений
                await self.app(scope, receive, send)
```

```

        return

    start = time.perf_counter()

    # Перехватываем send, чтобы измерить время до первого байта
    async def wrapped_send(message):
        if message["type"] == "http.response.start":
            elapsed = time.perf_counter() - start
            headers = message.get("headers", [])
            headers.append((b"x-process-time", str(elapsed).encode()))
            message["headers"] = headers
        await send(message)

    await self.app(scope, receive, wrapped_send)

```

### Преимущества:

- Работает на уровне ASGI, без промежуточных абстракций.
- Поддерживает streaming: `wrapped_send` вызывается для каждого чанка.
- Нулевые накладные расходы на буферизацию.
- Полный контроль над lifecycle запроса.

### Недостатки:

- Код сложнее: нужно работать с `scope`, `receive`, `send` напрямую.
- Нет готового объекта `Request` (его можно создать вручную, но это дорого).

## 4.7.4. Порядок выполнения middleware

Middleware в FastAPI образуют **лук** (onion): запрос проходит через них снаружи внутрь, ответ — изнутри наружу.

```

In [ ]: app.add_middleware(MiddlewareA)
        app.add_middleware(MiddlewareB)
        app.add_middleware(MiddlewareC)

# Порядок выполнения:
# Request: C -> B -> A -> Endpoint
# Response: Endpoint -> A -> B -> C

```

Это важно для:

- **Аутентификации:** должна быть ближе к endpoint'у, чтобы логирование уже видело идентификатор пользователя.
- **CORS:** обычно самый внешний слой.

- **Сжатия (GZip)**: должно быть после всех модификаций ответа.

### 4.7.5. Математическая подоплека: middleware как композиция функций

Middleware формируют **моноид** под композицией. Каждый middleware  $M$  — это функция высшего порядка:

$$M: (\text{Scope} \rightarrow \text{Receive} \rightarrow \text{Send} \rightarrow \text{Awaitable}) \rightarrow (\text{Scope} \rightarrow \text{Receive} \rightarrow \text{Send} \rightarrow \text{Awaitable})$$

или короче:

$$M: \text{ASGIApp} \rightarrow \text{ASGIApp}$$

Композиция middleware:

$$(M_3 \circ M_2 \circ M_1)(App) = M_3(M_2(M_1(App)))$$

Это ассоциативная операция с нейтральным элементом (тождественное middleware). В функциональном программировании такая структура называется **эндофунктор** (endofunctor) категории ASGI-приложений в себя.

## 4.8. Жизненный цикл приложения (Lifespan)

### 4.8.1. Проблема: ресурсы нужно инициализировать один раз

ML-сервис должен:

1. **При старте**: загрузить модель в GPU/CPU память, установить соединение с базой данных, подключиться к брокеру сообщений.
2. **При работе**: использовать эти ресурсы в endpoint'ах.
3. **При остановке**: корректно освободить память, закрыть соединения, дождаться завершения текущих запросов.

В старых фреймворках (Flask, Django) использовались хуки

`@app.before_first_request` — но это плохо: первый запрос платит за инициализацию задержкой.

ASGI ввёл протокол **Lifespan**: два события, `startup` и `shutdown`, которые происходят до приёма первого HTTP-запроса и после остановки

приёма новых соединений.

## 4.8.2. Lifespan в FastAPI: @asynccontextmanager

```
In [ ]: from contextlib import asynccontextmanager
        from fastapi import FastAPI

        ml_model = None # глобальная переменная для примера

        @asynccontextmanager
        async def lifespan(app: FastAPI):
            # Startup: выполняется до начала приёма запросов
            global ml_model
            ml_model = await load_ml_model() # тяжёлая асинхронная инициализация
            app.state.model = ml_model       # сохраняем в app.state
            print("Модель загружена")

            yield # <- здесь приложение начинает обрабатывать запросы

            # Shutdown: выполняется после остановки сервера
            await ml_model.dispose()
            print("Модель освобождена")

        app = FastAPI(lifespan=lifespan)
```

### Что происходит:

1. Uvicorn запускает приложение.
2. Отправляет lifespan-событие `startup`.
3. FastAPI входит в `lifespan`, выполняет код до `yield`.
4. `yield` передаёт управление Uvicorn — начинается приём HTTP/WebSocket.
5. Когда Uvicorn получает сигнал остановки (SIGTERM), он перестаёт принимать новые соединения, ждёт завершения текущих.
6. Отправляет lifespan-событие `shutdown`.
7. Код после `yield` выполняется — освобождение ресурсов.

## 4.8.3. Управление глобальным состоянием через `app.state`

`app.state` — это объект типа `Starlette.state.State`, хранящий произвольные атрибуты:

```
In [ ]: @app.get("/predict")
        async def predict(data: InputData):
```

```
model = app.state.model
result = await model.predict(data.features)
return {"prediction": result}
```

### Почему `app.state`, а не глобальные переменные?

- Глобальные переменные в Python — это переменные модуля. При использовании `multiprocessing` (несколько воркеров Uvicorn) каждый процесс имеет **свою** копию глобальных переменных.
- `app.state` явно привязан к экземпляру приложения и документирует намерение: «это состояние приложения».

## 4.8.4. Graceful shutdown: корректное завершение

Когда Uvicorn получает `SIGTERM`:

1. **Прекращает приём** новых TCP-соединений.
2. **Ждёт** завершения текущих HTTP-запросов (с таймаутом, по умолчанию 30 секунд).
3. **Закрывает** WebSocket-соединения.
4. **Вызывает** `lifespan.shutdown`.
5. **Завершает** процесс.

Важно для ML: если inference занимает 5 секунд, и в момент остановки обрабатывается 10 запросов — Uvicorn подождёт их завершения (в пределах таймаута). Запросы не обрываются посередине.

Можно настроить таймаут:

```
In [ ]: uvicorn main:app --timeout-graceful-shutdown 60
```

## 4.8.5. AsyncExitStack для сложной инициализации

Если нужно управлять несколькими ресурсами (БД, кэш, ML-модель), удобен `AsyncExitStack`:

```
In [ ]: from contextlib import AsyncExitStack

@asynccontextmanager
async def lifespan(app: FastAPI):
    async with AsyncExitStack() as stack:
        db = await create_db_pool()
        await stack.enter_async_context(db)
```



```

redis = await create_redis_client()
await stack.enter_async_context(redis)

model = await load_model()
app.state.db = db
app.state.redis = redis
app.state.model = model

```

```

yield # приложение работает

```

*# здесь AsyncExitStack автоматически закроет db и redis*

`AsyncExitStack` гарантирует, что все ресурсы будут освобождены в **обратном порядке** их создания, даже если одно из освобождений выбросит исключение.

## Итог модуля 4

| Концепция                 | Суть                                                                     | Практическое применение                          |
|---------------------------|--------------------------------------------------------------------------|--------------------------------------------------|
| <b>ASGI</b>               | Интерфейс <code>scope/receive/send</code> для асинхронных веб-приложений | Понимание, как Uvicorn взаимодействует с FastAPI |
| <b>Uvicorn</b>            | ASGI-сервер на <code>uvloop</code> + <code>httptools</code>              | Запуск продакшн-сервера                          |
| <b>Type-driven</b>        | Типы = валидация + документация                                          | Меньше кода, меньше багов                        |
| <b>Pydantic V2</b>        | Ядро на Rust, zero-copy                                                  | Валидация со скоростью, близкой к C              |
| <b>Exception Handler</b>  | Централизованная обработка ошибок                                        | Чистая бизнес-логика, единый формат ошибок       |
| <b>BaseHTTPMiddleware</b> | Простой, но опасен для streaming                                         | Только для простых middleware                    |
| <b>ASGI Middleware</b>    | Полный контроль, нет накладных расходов                                  | Streaming, логирование, rate limiting            |
| <b>Lifespan</b>           | <code>startup / shutdown</code> через <code>@asynccontextmanager</code>  | Загрузка ML-моделей, пулы соединений             |
| <b>app.state</b>          | Глобальное состояние приложения                                          | Хранение модели, доступной из endpoint'ов        |

В **Модуле 5** мы разберём Dependency Injection — систему `Depends`, которая позволяет строить масштабируемую архитектуру, не создавая глобальных переменных и не разбрасывая инициализацию ресурсов по endpoint'ам.